Operations on Processes

Lecture 7 — Creation, Termination, `fork/exec`

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Content Not Click

Welcome: Who Actually Runs `ls`?

The big question

You type `ls` at a shell prompt. A listing appears and the prompt comes back. The shell did not turn into `ls`, and `ls` did not exist a moment earlier. Where did the new process come from, and how did the shell know when it had finished?

- Every process except the first is created by another process.
- The shell creates a **child**, the child becomes `ls`, and the shell **waits**.
- Three system calls do the work: `fork`, `exec`, `wait` — plus `exit` at the end.

Running thread for today

We follow one shell command from creation to termination.

Roadmap: Four Questions About a Process's Life

1. **How is a process created?** — parents, children, and the process tree.
2. **What does `fork` return, and how many processes do we get?** — counting forks.
3. **How does a child become a different program?** — the `exec` family.
4. **How does a process end, and who cleans up?** — `exit`, `wait`, zombies, and orphans.

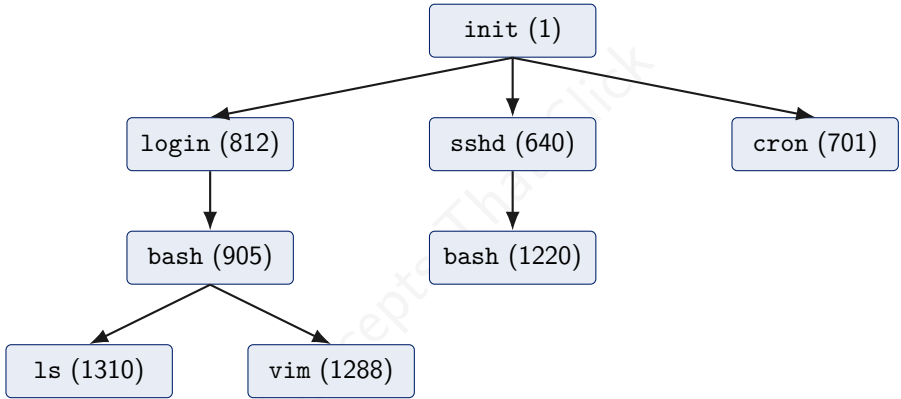
Lecture 5 gave us the PCB; Lecture 6 moved a process between states. Today we create and destroy the PCB itself.

Concepts That Click

A process is not conjured by the kernel out of nothing. It is copied from a parent that already exists.

Concepts Click

Parents, Children, and the Process Tree



Each arrow is one creation. Each process has a unique integer **process identifier** (pid). Pids are illustrative. Silberschatz et al., Ch. 3 §3.3, pp. 110–116.

What a Parent Must Decide

Definition

A **parent process** is a process that issues a creation call; the new process is its **child**. Repeated creation forms a **process tree** rooted at the first process.

- **Resources:** the child shares all, a subset, or none of the parent's resources (open files, memory).
- **Execution:** the parent continues concurrently with the child, *or* waits until some or all children terminate.
- **Address space:** the child is a duplicate of the parent (same program and data), *or* has a new program loaded into it.

UNIX answers the address-space question in two steps: `fork` duplicates, `exec` loads. Silberschatz et al., Ch. 3 §3.3.

Concept_That

`fork`: One Call, Two Returns

Definition

`fork()` creates a new process whose address space is a copy of the caller's. Both processes continue at the instruction after the call.

Who sees it	**Return value of `fork()`**
the child	0
the parent	the child's pid (a positive integer)
the parent, on failure	`-1`, and no child is created

The return value is the *only* way the two copies can tell themselves apart. A child can learn its own pid with `getpid()` and its parent's with `getppid()`.

Worked Example: The Canonical `fork/exec/wait`

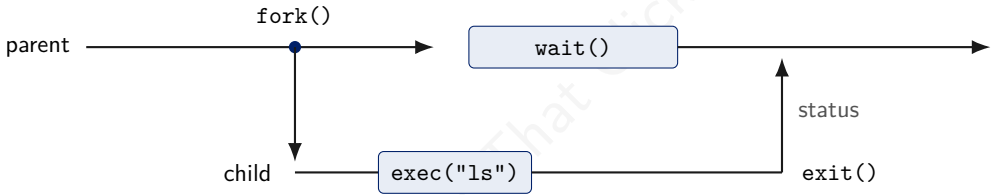
```
pid_t pid = fork();
if (pid < 0) {              /* error  */
    perror("fork");
    exit(1);
} else if (pid == 0) {      /* child  */
    execlp("ls", "ls", NULL);
    perror("exec");  /* on failure */
    exit(1);
} else {                    /* parent */
    wait(NULL);
    printf("Child complete\n");
}
```

1. `fork` returns twice.
2. The child takes the `pid == 0` branch and replaces itself with `ls`.
3. The parent takes the `else` branch and blocks in `wait`.
4. `ls` finishes; `wait` returns; the parent prints.

Same structure as Silberschatz et al., Ch. 3 §3.3, pp. 110–116.

Concepts That Click

The Same Program as a Timeline



- The parent is blocked in `wait` while the child runs.
- The child's `exit` wakes the parent and delivers a status.

Concepts That Click

Two Copies, Two Variables

- After `fork`, parent and child have *separate* address spaces with identical contents.
- A write in one is invisible to the other.

Worked example (verified by running it)

```
int x = 10; pid = fork();
child: x += 5; printf("child x=%d", x);   prints child x=15
parent: wait(NULL); x -= 5; printf("parent x=%d", x);   prints parent x=5
```

- Same variable name, same virtual address, *different* physical copies.
- What *is* shared: open file descriptors (both point to the same open-file entries).

Socratic Check 1

Question

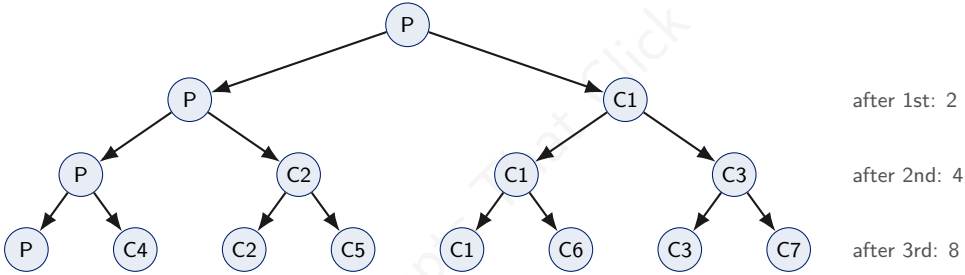
A child process wants to tell its parent the answer to a computation. Can it simply store the answer in a global variable the parent will read after `wait`?

- No. The global lives in the child's copy of the address space.
- The parent's copy still holds the old value.
- The child can return a small integer through its `exit` status; anything larger needs inter-process communication (Lectures 14–15).

Every fork doubles the number of processes that execute the next line. Counting them is a tree, not a guess.

Concepts Click

Three `fork` Calls in a Row



`fork(); fork(); fork(); printf("hi\n");` prints **8** lines. In general, n sequential forks give 2^n processes, of which $2^n - 1$ are new children. Verified: $n = 3$ gives 8 lines, $n = 4$ gives 16.

Concepts Tech Click

Worked Example: `printf` Inside the Loop

Code

```
for (i = 0; i < 3; i++) { fork(); printf("A\n"); }
```

1. Iteration $i = 0$: 1 process forks into 2; each prints once: **2** lines.
2. Iteration $i = 1$: 2 processes fork into 4; each prints: **4** lines.
3. Iteration $i = 2$: 4 processes fork into 8; each prints: **8** lines.

Answer

$2 + 4 + 8 = 14$ lines of `A`, from 8 processes in total. (Verified by running it.)

Short-Circuit Operators Prune the Tree

- In `a && b`, `b` runs only if `a` is non-zero; in `a || b`, `b` runs only if `a` is zero.
- `fork()` is non-zero in the parent and zero in the child, so the operators decide *which copy* forks again.

Trace: `(fork() && fork()) || fork(); printf("y\n");`

1. First `fork`: parent P (non-zero), child C1 (zero).
2. P evaluates the second `fork`: P (non-zero), C2 (zero). P's `&&` is true, so P skips the `||`.
3. C1's `&&` is false, so C1 runs the third `fork`: C1 and C3.
4. C2's `&&` is false, so C2 also forks: C2 and C4.

Processes: P, C1, C2, C3, C4 — **5** lines. (Verified.)

The Buffer That Gets Copied

- `printf` writes into a user-space buffer (Lecture 3); the `write` system call comes later.
- `fork` copies the *whole* address space — including any bytes still sitting in that buffer.

Trap

`printf("hello "); fork(); printf("world\n");`
prints `hello world` **twice**: the unflushed `"hello "` was duplicated into the child, and each copy flushes its own buffer. (Verified.)

- Fix: call `fflush(stdout)` before `fork`.
- Lesson: `fork` copies *state*, not just code.

Socratic Check 2

Question

`if (fork() && fork()) fork(); printf("x\n");` How many lines are printed?

- First `fork`: P and C1. C1 sees 0, so the `&&` is false: C1 skips the body.
- P runs the second `fork`: P and C2. C2 sees 0: skips the body.
- P's condition is true: P runs the body's `fork`, creating C3.
- Processes P, C1, C2, C3: **4** lines. (Verified.)

Concepts That Work

A copy of the shell is useless until it stops being the shell. exec throws the old program away and keeps the process.

exec: Same Process, New Program

Definition

An exec call replaces the calling process's memory image — code, data, heap, stack — with a new program loaded from an executable file, and starts it at its entry point.

- The **pid stays the same**; the parent relationship stays the same.
- Open file descriptors stay open (unless marked close-on-exec).
- On success exec **never returns**: the code that called it no longer exists.
- On failure it returns `-1`; the old program continues.

Bovet & Cesati, Ch. 20, pp. 808–834 (execve).

Reading the exec Names

Letter	Meaning
`l`	arguments as a *list*: `execl("/bin/ls", "ls", "-l", NULL)`
`v`	arguments as a *vector* (array): `execv(path, argv)`
`p`	search the `PATH` for the file: `execlp("ls", ...)`
`e`	pass an explicit *environment*: `execle`, `execve`

- All are library wrappers; on Linux the one system call underneath is `execve`.
- Argument lists end with `NULL`; `argv[0]` is the program's own name by convention.

Concepts That Click

Worked Example: Code After exec

Code

child: `execlp("echo", "echo", "from-exec", NULL); printf("never\n");`
parent: `wait(NULL); printf("parent\n");`

1. The child's image is replaced by `echo`, which prints `from-exec` and exits.
2. The `printf("never")` line was part of the old image; it is gone and never runs.
3. The parent's `wait` returns; it prints `parent`.

Output (verified)

`from-exec`, then `parent` — two lines, never three.

Why Split Creation Into `fork` + `exec`?

- Between the two calls the child is still running *the shell's code* — but in its own process.
- It can rearrange its own environment before the new program starts.

The shell running `ls > out.txt`

1. Shell: `fork`.
2. Child: `fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC)`.
3. Child: `dup2(fd, 1)`; `close(fd)` — standard output now refers to the file.
4. Child: `execlp("ls", "ls", NULL)` — `ls` writes to descriptor 1 and never knows about the file.
5. Shell: `wait`.

Redirection, pipes, and changed directories all happen in this gap.

Is Copying the Whole Address Space Not Wasteful?

- A child that calls `exec` immediately discards the copy `fork` just made.
- Modern kernels use **copy-on-write**: parent and child share the pages read-only, and a page is copied only when one of them writes it.
- `fork` then costs roughly a copy of the page tables and the PCB, not of the memory.
- Linux also provides `vfork` and `clone` as variants of process creation.

Where this returns

Copy-on-write is a virtual-memory mechanism; we study it in Lecture 26.

Bovet & Cesati, Ch. 3, pp. 79–130 (`fork`, `clone`, `vfork`, copy-on-write).

Another Design: One-Step Creation

UNIX: two steps	**Windows: one step**
`fork` duplicates, then `exec` loads	`CreateProcess` loads a named program into a new process
Child customizes itself between the calls	Customization passed as parameters to the single call
Parent waits with `wait`	Parent waits with `WaitForSingleObject`

Both answer the same three parental decisions (resources, execution, address space); they differ in *where* the answers are supplied. Silberschatz et al., Ch. 3 §3.3, pp. 110–116.

Creation is half the life cycle. When a process ends, something must collect what it leaves behind.

Concepts Clinic

Normal Termination: `exit` and `wait`

- A process terminates after its last statement or by calling `exit(status)`.
- The kernel releases its memory, open files, and other resources.
- The small integer `status` is kept for the parent.
- The parent collects it with `wait`, which returns the pid of the terminated child.

Worked example (verified)

Child: `exit(7);` Parent: `waitpid(pid, &st, 0);`
`WIFEXITED(st)` is true and `WEXITSTATUS(st)` is 7.

`waitpid` waits for one named child; `wait` for any.

When a Parent Terminates a Child

A parent may end a child with a system call (`kill` on UNIX, `TerminateProcess` on Windows). Typical reasons:

- The child has exceeded its allocated resources.
- The task assigned to the child is no longer required.
- The parent is exiting, and the system does not allow a child to continue without its parent.

Cascading termination

On systems with the third rule, terminating a parent terminates all its descendants; the operating system initiates it.

Silberschatz et al., Ch. 3 §3.3, pp. 110–116. Only a parent (or a suitably privileged user) may kill a process.

Zombies and Orphans

Zombie

A process that has terminated but whose parent has *not yet* called `wait`. Its resources are freed, but its PCB entry (pid, exit status) remains so the parent can collect it.

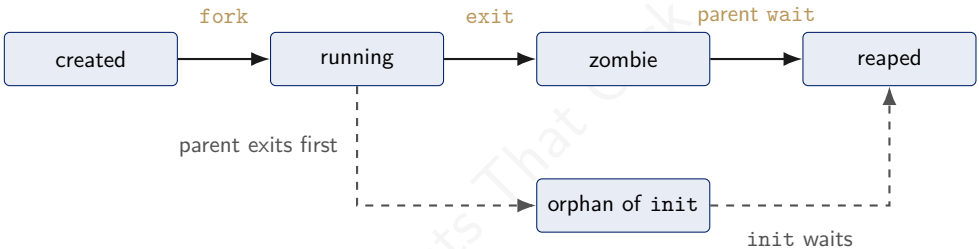
Orphan

A process whose parent terminated *without* waiting for it. UNIX re-parents it to `init` (pid 1), which calls `wait` periodically.

- Every child is briefly a zombie; the problem is a parent that never waits.
- Many zombies exhaust the pid table even though they use no memory.

Standard UNIX treatment; Bovet & Cesati, Ch. 3 (destroying processes), pp. 79–130.

The Life Cycle at the Boundary



- Solid path: the normal case. Dashed path: the parent leaves first.
- "Zombie" is a bookkeeping state, not a running program.

Worked Example: The Shell Loop

1. Print the prompt and read a command line, e.g. `ls -l`.
2. `pid = fork()`.
3. Child (`pid == 0`): set up redirection if any, then `execvp("ls", argv)`; if exec returns, print an error and `exit(127)`.
4. Parent: `waitpid(pid, &status, 0)` — unless the line ended with `&`, in which case it returns to step 1 immediately.
5. Repeat.

System calls per foreground command

`fork` (shell) → `execvp` (child) → `exit` (child) → `wait` (shell): **4** process-control calls, the same category from Lecture 3.

Socratic Check 3

Question

A server forks one child per request and never calls `wait`. The children finish in milliseconds. What accumulates, and what is the fix?

- Every finished child becomes a zombie: its pid and status stay in the process table.
- Memory is released, but pids run out after enough requests.
- Fix: reap the children — call `wait`/`waitpid` (e.g. in a loop with `WNOHANG`) as they finish.

Concept Thinking

Common Misconceptions

Wrong

- "`fork` returns the child's pid to the child."
- "Parent and child share variables."
- "`exec` creates a new process."
- "Code after a successful `exec` runs when it finishes."
- "A zombie is still using CPU and memory."

Right

- The child gets `0`; the parent gets the child's pid.
- They have separate copies; only descriptors are shared.
- `exec` replaces the image; the pid is unchanged.
- A successful `exec` never returns.
- A zombie holds only its pid and exit status.

Takeaways

- Processes form a **tree**: every process but the first has a parent.
- `fork` returns **twice**: `0` in the child, the child's pid in the parent, `-1` on failure.
- n sequential forks give 2^n processes; short-circuit operators and loops change the count — draw the tree.
- `exec` keeps the process and replaces the program; it returns only on failure.
- `exit` delivers a status; `wait` collects it. An unreaped child is a **zombie**; a parentless one is an **orphan** adopted by `init`.

Concepts That Click

Next: CPU Scheduling — Concepts, FCFS and SJF

- Today: how processes are born, transformed, and reaped.
- Next lecture: once many processes are ready, which one gets the CPU? Scheduling criteria (TAT, WT, RT) and the first two policies.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 5 §5.1–5.3 [3]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 9 §9.1–9.2 [4].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 3 §3.3 (operations on processes: creation, termination, `fork`/`exec`, pp. 110–116) [3].
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 3 §3.4 (process control, p. 135) [4].
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 3 (creating and destroying processes, pp. 79–130) and Ch. 20 (program execution, pp. 808–834) [2].
- M. J. Bach, *The Design of the UNIX Operating System* — chapter-level reference for the UNIX process view [1].

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/` (Bach is not indexed).

Concepts that C k

References I


Maurice J. Bach.
The Design of the UNIX Operating System.
Prentice Hall of India, 1986.


Daniel P. Bovet and Marco Cesati.
Understanding the Linux Kernel.
O'Reilly Media, 3rd edition, 2006.


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click